

Program Repair and Automated Debugging with Code LLMs

Abstract

The advent of Large Language Models (LLMs) has precipitated a paradigm shift in Automated Program Repair (APR) and automated debugging. Moving beyond traditional search-based and template-driven methodologies, contemporary approaches leverage the generative capabilities of code-specialized transformers to synthesize patches, localize faults, and reason about program semantics. This survey provides a comprehensive synthesis of 300 recent academic works, categorizing the field into distinct methodological families: retrieval-augmented generation, agentic and multi-agent workflows, execution-guided reasoning, fine-tuning strategies, and formal verification integration. We analyze the evolution from zero-shot prompting to complex, tool-augmented reasoning loops that incorporate static analysis, dynamic tracing, and compiler feedback. Furthermore, we examine the critical role of datasets and benchmarks in evaluating these systems, highlighting the transition from function-level benchmarks like Defects4J to repository-scale challenges such as SWE-bench. Despite significant advancements in patch plausibility and correctness, challenges remain regarding computational efficiency, overfitting to test suites, robustness against syntactic perturbations, and the handling of multi-hunk, cross-file defects. This paper delineates the current state-of-the-art, synthesizes empirical findings across diverse architectural choices, and outlines future research directions for building reliable, autonomous debugging systems.

1. Introduction

Automated Program Repair (APR) has long been a cornerstone of software engineering research, aiming to automatically generate source code modifications that correct software defects. Historically, APR techniques were categorized into search-based generate-and-validate approaches, constraint-based synthesis, and template-based methods [1], [2]. While effective for specific bug patterns, these traditional methods often struggled with the vast search space of possible patches and lacked the semantic understanding required for complex, non-local fixes. The emergence of Large Language Models (LLMs) trained on massive code corpora has fundamentally altered this landscape, offering unprecedented capabilities in code generation, understanding, and reasoning [3], [4], [5].

The integration of LLMs into APR has evolved rapidly from simple zero-shot prompting to sophisticated, multi-stage pipelines. Early studies demonstrated that off-the-shelf models like Codex could fix a significant number of bugs in benchmarks such as QuixBugs and Defects4J without task-specific fine-tuning, often outperforming specialized neural repair tools [4], [5], [6]. However, naive generation frequently results in syntactically invalid code or patches that overfit to available test cases [7], [8]. Consequently, recent research has focused on augmenting LLMs with external tools, retrieval mechanisms, and iterative feedback loops to ground generation in program semantics and execution behavior.

This survey synthesizes evidence from 300 academic papers to provide a structured overview of the field. We organize the discourse around several key methodological axes. First, we examine **Retrieval-Augmented Generation (RAG)**, where models are conditioned on relevant bug-fix pairs or project context to guide repair [9], [10], [11]. Second, we explore **Agentic and Multi-Agent Systems**, which decompose debugging into planning, localization, and repair sub-tasks performed by specialized agents or iterative loops [12], [13], [14]. Third, we discuss **Execution-Guided and Tool-Augmented Reasoning**, where models utilize compiler errors, runtime traces, and debugger interactions to refine hypotheses [15], [16], [17]. Fourth, we analyze **Training and**

Fine-Tuning Strategies, including parameter-efficient adaptation, multi-objective learning, and synthetic data generation [18], [19], [20]. Finally, we address **Evaluation and Challenges**, covering benchmark construction, robustness issues, and the gap between plausible and correct patches [21], [22], [23].

By synthesizing these diverse strands of research, this paper aims to clarify the technical mechanisms driving modern LLM-based repair, identify persistent limitations, and propose a roadmap for future advancements in autonomous software debugging.

2. Method

The methodological landscape of LLM-based program repair is diverse, ranging from direct code generation to complex neuro-symbolic frameworks. We categorize existing approaches into six primary families: Retrieval-Augmented Generation, Agentic and Multi-Agent Workflows, Execution-Guided and Dynamic Analysis, Training and Adaptation Strategies, Formal and Static Analysis Integration, and Specialized Repair Domains.

2.1 Retrieval-Augmented Generation (RAG) for Contextual Repair

A dominant theme in recent APR research is the augmentation of LLMs with non-parametric memory through retrieval mechanisms. The core insight is that LLMs, despite their vast pre-training, may lack specific project context or rare fix patterns, which can be supplied via retrieval [9], [3].

2.1.1 Hybrid Retrieval Strategies Effective retrieval requires balancing lexical similarity with semantic relevance. **RAP-Gen** introduces a hybrid retriever combining sparse BM25 and dense Dense Passage Retrieval (DPR) using CodeT5 encoders to fetch relevant bug-fix pairs, which are then injected into the prompt via special tokens to guide generation [9]. Similarly, **SelRepair** employs a dual patch retriever that fuses semantic similarity (via UnixCoder) with static structure dependency (AST vectors), demonstrating that combining semantic and structural retrieval significantly outperforms semantics-only approaches [10]. **RELREPAIR** extends this by executing a three-stage incremental retrieval process: base repair, signature-based retrieval of function signatures, and snippet-based retrieval of intra- and inter-file code, achieving a 112.8% improvement over baseline ChatGPT prompting [11].

2.1.2 Project-Specific and Repository-Level Context Repairing bugs in large repositories often requires context beyond the immediate function. **RepoRepair** generates hierarchical documentation via LLMs to abstract repository semantics, enabling accurate file and function localization without agents [24]. **KGCompass** constructs a repository-aware knowledge graph linking issues and pull requests to code entities, utilizing path-guided generation to resolve 45.67% of issues in SWE-Bench Lite [25]. **FitRepair** emphasizes project-specific token learning by fine-tuning with aggressive masking (50%) on buggy projects, forcing the model to internalize project-specific identifiers and conventions, thereby fixing 15 more bugs than AlphaRepair on Defects4J [26]. **SuperCoder2.0** implements a hierarchical search combining RAG with repository file maps and AST parsing to localize methods before regenerating entire methods, achieving a 34% resolution rate on SWE-bench Lite [27].

2.1.3 Edit-Driven and Pattern-Based Retrieval Beyond static code similarity, some approaches retrieve based on edit trajectories. **LSGEN** employs edit-driven retrieval by vectorizing

code editing processes ($h_p = h_{c_r} - h_{c_w}$) to match buggy code with similar repair trajectories, improving code accuracy by 50.86% over baselines [28]. **RAPGen** extracts natural language fix instructions from historical commits rather than raw code examples, using AST-based abstraction to retrieve transformation instructions for performance bug fixing [29]. **Peer-aided Repairer (PaR)** selects optimal reference code from peer solutions using a weighted metric combining lexical, syntactic, and semantic similarities, boosting repair rates for student assignments by nearly 20% [30].

2.2 Agentic and Multi-Agent Workflows

The complexity of real-world debugging has spurred the development of agentic systems that decompose tasks into planning, localization, and repair, often involving multiple specialized agents or iterative self-reflection loops.

2.2.1 Single-Agent Iterative Loops Single-agent frameworks often rely on ReAct (Reasoning and Acting) patterns to iteratively refine patches. **ChatRepair** initializes a conversational loop where test failure logs and compilation errors are fed back into the context, allowing the model to iteratively generate and validate patches, fixing 114 bugs on Defects4J [31]. **ThinkRepair** automates the collection of Chain-of-Thought (CoT) reasoning paths by verifying fixes against test suites to build a knowledge pool, which is then used for few-shot selection, outperforming 12 SOTA APRs [32]. **CIGAR** optimizes token usage by employing a “search reboot” mechanism with high temperature to escape local minima and a “patch multiplication” strategy to synthesize diverse candidates, reducing token costs by 73% compared to ChatRepair [33]. **Debug2Fix** introduces a specialized Debug Subagent that encapsulates low-level debugger interactions (JDB/PDB), enforcing a workflow where the main agent must utilize debugging tools before editing, improving pass rates by 21.8% [34].

2.2.2 Multi-Agent Collaboration Multi-agent systems distribute cognitive load across specialized roles. **PATCH** decomposes bug fixing into reporting, diagnosis, generation, and verification stages, deploying Tester, Developer, and Reviewer agents that simulate collaborative workflows, surpassing GPT-4 by 14.01% on the BFP dataset [13]. **AdverIntent-Agent** utilizes a three-agent framework where one agent infers adversarial program intents, another generates differential test cases, and a third performs repair, effectively mitigating overfitting by aligning with true developer intent [35]. **RAMP (Ruby Automated Multi-agent Program repair)** orchestrates four agents (Feedback Integrator, Test Designer, Programmer, Refiner) to handle Ruby-specific repairs, demonstrating that test generation and self-reflection are critical drivers for success [36]. **Trace-Coder** employs three agents (Instrumentation, Analysis, Repair) and a Historical Lesson Learning Mechanism to prevent recurring error patterns, achieving an 82% pass@1 rate on ClassEval [37]. **EVIACT** implements a four-stage pipeline with evidence-driven guardrails (retrieval scaffold, compile gate) to reduce mislocalization cascades, resolving 38% of SWE-bench Lite issues [38].

2.2.3 Search-Based Agentic Strategies Integrating search algorithms with LLMs enhances exploration of the patch space. **APRMCTS** structures patch search as a tree using Monte Carlo Tree Search (MCTS), balancing exploration and exploitation via Upper Confidence Bound for Trees (UCT), fixing 201 bugs on Defects4J [39]. **CodePilot** combines Qwen3 dual-mode reasoning with MCTS for patch generation, utilizing execution feedback as dynamic reward signals to explore diverse solution paths [40]. **T3** constructs multiple independent reasoning trees in parallel (Forest of Thinking) for cause analysis, enhancing hypothesis diversity compared to linear chains

[41]. **BESTER** employs a best-first tree search guided by unit test pass rates and self-reflection, outperforming temperature sampling and Reflexion baselines [42].

2.3 Execution-Guided and Dynamic Analysis

Purely static generation often fails to capture runtime behaviors. Integrating dynamic execution traces, debugger outputs, and runtime states provides grounded feedback for repair.

2.3.1 Trace-Augmented Generation Injecting execution traces into prompts allows models to observe variable states and control flow. **TraceFixer** concatenates buggy code with partial execution traces and desired target states, constraining the search space and improving success rates by 13% over code-only baselines [16]. **DynaFix** utilizes non-intrusive bytecode instrumentation (ByteTrace) to capture fine-grained runtime states, employing a layered progressive repair strategy that outperforms single-shot injection [43]. **LDB (Large Language Model Debugger)** segments programs into basic blocks and captures intermediate variable states at block exits, prompting the LLM to verify simpler units against task descriptions, achieving 95.1% pass@1 on HumanEval [44]. **Print Debugging** inserts print statements to capture intermediate states, revealing inconsistencies invisible to static analysis and improving accuracy on medium-difficulty LeetCode problems by 17.9% [45].

2.3.2 Debugger Interaction and Scientific Debugging Emulating human debugging processes via tool use yields robust results. **AuToSD** emulates the scientific debugging cycle (hypothesis, experiment, observation) by prompting an LLM to generate debugger commands and observe results, outperforming Codex with fewer candidates [15]. **ChatDBG** grants LLMs autonomy to execute debugger commands (e.g., inspect variables, navigate stack) via function calling, constructing enriched stack traces that improve success rates to 85% on student labs [17]. **VulDebugger** orchestrates an agent to execute Proof-of-Concept inputs, extract crash-free constraints, and inspect runtime states via LLDB, achieving 60% repair accuracy on real-life projects [46]. **IntDiagSolver** implements an interactive cycle where the LLM self-plans targeted queries to complete contextual information for environment-related crash bugs, improving repair accuracy by 53.3% [47].

2.3.3 Test-Driven and Feedback Loops Test cases serve as critical oracles for guiding repair. **ContrastRepair** constructs contrastive test pairs by minimizing the distance between failing and passing tests, isolating root causes more effectively than standard feedback, fixing 143 bugs on Defects4J [48]. **Repair-R1** formulates repair as latent variable inference, using Reinforcement Learning to sequentially generate discriminative test cases before patches, improving repair success by 48.29% [49]. **DePro** generates a brute-force reference solution and stress-tests user code to isolate failure-inducing edge cases, reducing debugging attempts by 64% [50]. **FeedbackEval** benchmarks show that test feedback yields the highest repair success due to explicit error localization, outperforming simulated human feedback [51].

2.4 Training and Adaptation Strategies

While zero-shot prompting is powerful, fine-tuning and adaptation strategies tailor models to the specific objectives of program repair.

2.4.1 Parameter-Efficient Fine-Tuning (PEFT) Full fine-tuning is often computationally prohibitive and risks catastrophic forgetting. **RepairLLaMA** integrates Low-Rank Adaptation

(LoRA) into CodeLlama, using region-based fault localization encoding to support multi-location fixes, outperforming full fine-tuning by 46 bugs [19]. **PAFT (Preservation-Aware Fine-Tuning)** computes token-level alignment between buggy and fixed code, applying weighted loss functions to preserve unchanged tokens, yielding a 65.6% relative gain in pass@1 [18]. **MOREpair** optimizes a joint loss function balancing code generation and rationale synthesis using QLoRA, narrowing the gap with GPT-4 [52]. **APR-Instruction** demonstrates that PEFT methods like (IA)³ can fix up to 31.3% more bugs than no fine-tuning while reducing GPU memory usage by half [53]. **Federated LLM-Based APR** employs QLoRA in a federated setting to preserve data privacy while achieving performance comparable to centralized learning [54].

2.4.2 Synthetic Data and Self-Supervised Learning Data scarcity and quality are addressed via synthetic generation and self-supervised loops. **MUFIN** employs a back-translation loop with neural breaker-fixer pairs and code critics to generate diverse functional bugs, improving repair counts by 12 on Defects4J [55]. **Break-It-Fix-It (BIFI)** iteratively trains a fixer and a breaker, using a critic to verify fixes on real bad data and generating realistic bad code for augmentation [56]. **SelfAPR** generates synthetic bugs via AST perturbations on past project versions and incorporates test execution diagnostics into the input, outperforming supervised approaches [57]. **SWE-Synth** masks code components in verified repositories and prompts LLMs to re-implement them, filtering variants via test execution to create high-quality synthetic data [58]. **Cross-Model Synthetic Data Generation** uses frontier models to generate buggy-fixed pairs across 12 languages, filtering via cross-model consensus to mitigate bias [20].

2.4.3 Reinforcement Learning and Preference Optimization Aligning model outputs with correctness and minimal edit distance is achieved via RL. **RePair** trains a reward model on automated execution feedback to provide process-based supervision, aligning better with human debugging cycles [59]. **SecureCode** utilizes Proximal Policy Optimization (PPO) with a composite reward function integrating CodeBLEU and BERTScore to guide security repairs [60]. **COF-FEEPOTS** aligns feedback generation preferences with actual edit success rates via Direct Preference Optimization (DPO), significantly improving error localization [61]. **PRrepair** employs Edit-Aware Group Relative Policy Optimization (EA-GRPO) with dynamic edit penalties to optimize for minimal sufficient edits, improving fix_1@1 by 20.95% [62]. **BitsAI-Fix** uses progressive RL shifting from compilability checks to golden-patch similarity for lint error resolution [63].

2.5 Formal and Static Analysis Integration

Combining neural generation with symbolic reasoning ensures semantic correctness and reduces hallucinations.

2.5.1 Static Analysis-Guided Repair Static analyzers provide precise bug locations and constraints. **InferFix** integrates the Infer static analyzer for bug detection and retrieves semantically similar historic fixes to augment prompts, achieving 76.8% exact match accuracy [64]. **GIANTREPAIR** extracts concrete modifications from LLM patches into structural skeletons and uses static analysis for context-aware instantiation, fixing 171 bugs on Defects4J [65]. **dcFix** executes static data flow analysis to verify API attributes against specifications, injecting unsatisfied constraints into prompts to improve REST API repair [66]. **DeepCode AI Fix** uses provenance-guided Hierarchical Delta Debugging to isolate vulnerability-relevant code, reducing input size and focusing attention on causal paths [67]. **StaticFixer** employs static-analysis witnessing to identify sanitizers in safe programs, generating unsafe-safe pairs for strategy synthesis [68].

2.5.2 Formal Verification and Symbolic Execution Formal methods provide mathematical guarantees of correctness. **ESBMC-AI** integrates Bounded Model Checking (BMC) with LLMs in a closed loop, using counterexamples to guide iterative repair of memory safety properties, achieving 90.4% accuracy on buffer overflows [69]. **Assured Automatic Programming (ProofRover)** partitions verification conditions into hard and soft intents, co-evolving code and specifications to ensure conformance [70]. **Counterexample Guided Program Repair** uses MaxSAT solvers to identify minimal faulty statement sets and generate program sketches for LLM synthesis [71]. **AutoCodeSherpa** generates symbolic bug explanations as Hoare triples implemented via property-based tests, providing verifiable characterizations of bugs [72]. **C2SAFERRUST** utilizes static analysis to decompose C code into translation units, guiding LLMs to transform unsafe Rust skeletons into safe code [73].

2.5.3 Neuro-Symbolic and Grammar-Constrained Approaches Constraining generation to valid grammars or edit scripts improves syntactic validity. **Recoder** employs a syntax-guided edit decoder generating AST edits via a provider/decider architecture, guaranteeing syntactic correctness [74]. **NSEdit** formulates repair as neural-symbolic script generation using a regular edit grammar and pointer networks for location selection [75]. **GLAD** shifts from NMT to generative synthesis with grammar-constrained beam search to repair omission faults, ensuring type compatibility [76]. **LaMirage** combines syntax-guided enumeration with neural localization for low-code formula languages, fixing non-local errors without search space explosion [77]. **Clover** orchestrates RTL repair via a main agent and SMT-based symbolic solvers for low-level template repairs [78].

2.6 Specialized Repair Domains and Tasks

APR research has expanded beyond general-purpose bugs to specialized domains and task formulations.

2.6.1 Security and Vulnerability Repair Security repair requires addressing specific vulnerability classes. **VRepair** employs transfer learning from generic bug fixes to vulnerability fixing, improving sequence accuracy by 14% [79]. **LLM4CVE** integrates CVE metadata and iterative feedback with LoRA fine-tuning to improve semantic similarity to ground truth fixes [80]. **SecureFixAgent** integrates Bandit static analysis with local LLMs in a detect-repair-validate loop, achieving 87.83% fix accuracy [81]. **Repairing Regex Vulnerabilities** decouples localization from repair using symbolic analysis to identify vulnerable subpatterns, improving repair rates by 15.4% [82]. **Neural Transfer Learning** demonstrates that bug fixing and vulnerability fixing share similarities, enabling knowledge transfer [79].

2.6.2 Educational and Student Code Repair Repairing student code involves handling syntax errors and preserving pedagogical intent. **Automated Repair of Uncompilable Student Code** evaluates LLMs on syntax-only repairs, finding that model architecture significantly impacts logic preservation [83]. **MMAPR** executes a two-phase pipeline separating syntax and semantic repair, utilizing structure-based chunking to minimize spurious edits [84]. **FastFixer** integrates repair-oriented fine-tuning with modification-focused masking to prioritize loss on changed segments, accelerating inference by 16.67x [85]. **RePurr** employs an elitist genetic algorithm with assertion-level fault localization to repair block-based Scratch programs [86]. **PyFiXV** generates high-precision feedback for syntax errors using a generate-explain-validate pipeline, simulating student validation [87].

2.6.3 Multilingual and Low-Resource Repair Extending APR to diverse languages addresses global software needs. **MDEVAL** constructs a massively multilingual benchmark across 20 languages, revealing that closed-source models significantly outperform open-source ones in multilingual debugging [88]. **RING** conceptualizes repair as generation, leveraging few-shot prompts with smart retrieval based on error diagnostic similarity for multilingual tasks [89]. **MultiMend** fine-tunes CodeT5 on multilingual pairs and employs retrieval-augmented generation to embed relevant lines from the buggy file, significantly outperforming baselines on Codeflaws [90]. **Investigating Transferability** shows that transferring code completions alongside rationales boosts performance in low-resource languages like Perl [91].

2.6.4 Hardware and Domain-Specific Languages APR techniques are adapted for hardware description languages and DSLs. **MEIC** orchestrates a dual-agent loop with rollback mechanisms for RTL debugging, achieving 93% syntax fix rates [92]. **VeriDebug** unifies bug detection and correction via contrastive embedding and guided correction for Verilog, improving fix accuracy by 53.4% [93]. **HDLdebugger** generates synthetic buggy-correct HDL pairs via reverse engineering and uses RAG for debugging, outperforming GPT-4 [94]. **EqFix** fixes LaTeX equation errors using programming-by-examples with pattern matching, solving 80.9% of cases [95]. **On Repairing Quantum Programs** applies conversational LLMs to Qiskit code, showing that detailed bug descriptions significantly improve repair rates [96].

2.7 Fault Localization and Context Engineering

Accurate fault localization is a prerequisite for effective repair. LLMs are increasingly used to enhance or replace traditional localization techniques.

2.7.1 LLM-Based Fault Localization **Toggle** decouples localization and repair, using distinct LLMs to predict token-granular buggy indices, minimizing generation overhead [97]. **RGFL** replaces standard retrieval ranking with a two-stage reasoning module where an LLM generates natural language explanations for candidate relevance, improving file-level Hit@1 by 13.6% [98]. **BugCerberus** executes a hierarchical process with customized LLMs for file, function, and statement granularity, integrating program slicing to surpass SOTA baselines by 16.5% [99]. **FLARE** aligns LLM subword probabilities to AST-aligned lexical units to generate fine-grained diagnostic feedback, improving pass@1 by 8.5% [100]. **TokenRepair** computes context-aware token uncertainty fluctuations to localize suspicious tokens, guiding iterative refinement [101].

2.7.2 Context Expansion and Granularity The granularity of context provided to LLMs critically impacts performance. **On the Role of Fault Localization Context** demonstrates that file-level expansion yields dominant gains, with LLM-based retrieval outperforming rule-based methods at lower token costs [102]. **A Study on the Impact of Fault localization Granularity** shows that function-level granularity yields the highest resolve rates (~48%) on SWE-Bench Mini under perfect localization assumptions [103]. **Out of Context** finds that repair success correlates positively with context size up to 56 lines, particularly for bugs requiring identifier changes [104]. **ScanFix** employs a dedicated neural scanner to extract far-away identifier ingredients, improving repair success for distant dependencies [105].

3. Challenges and Outlook

Despite remarkable progress, LLM-based APR faces significant challenges that hinder widespread deployment in industrial settings.

3.1 Evaluation Bottlenecks and Benchmark Leakage

A critical issue is the reliability of evaluation benchmarks. **RepairBench** highlights that execution-based metrics are more reliable than static matching, yet many benchmarks suffer from data leakage where test cases or fixes appear in pre-training data [106]. **EvalGPTFix** constructs a clean benchmark using post-cutoff data, revealing that while ChatGPT fixes 72.19% of bugs, performance drops significantly on contaminated datasets [107]. **HEJ-Robust** demonstrates that fine-tuned models exhibit severe performance degradation (50-57% drop) under minor syntactic perturbations like variable renaming, indicating overfitting to surface forms rather than semantic understanding [108]. **DEFECTS4J-TRANS** confirms this, showing an average 49.48% drop in correct patches on semantically equivalent transformed code [22]. Furthermore, **PRECISE DEBUGGING BENCHMARKING** reveals that frontier LLMs achieve high unit-test scores but low edit precision, suggesting a reliance on code regeneration rather than precise fault targeting [23].

3.2 Computational Cost and Efficiency

Agentic and iterative approaches incur substantial computational costs. **ChatRepair** and similar conversational methods can require extensive token usage, with costs scaling linearly with conversation length [31]. **CIGAR** addresses this by optimizing prompt design and employing search rebooting, reducing token costs by 73% [33]. **Refine** reports an average cost of \$6.59 per issue due to iterative LLM calls [109]. **RGFL** notes a 3.7x increase in computational cost due to multiple LLM calls per candidate [98]. **NARRepair** and **FastFixer** explore non-autoregressive and speculative decoding strategies to accelerate inference, yet validation time often remains a bottleneck [110], [85].

3.3 Overfitting and Patch Correctness

The “plausible but incorrect” patch problem persists. **A Comprehensive Study of Code-removal Patches** finds that many successful patches are actually code-removals that pass weak or flaky tests, with only 2.95% representing correct fixes in CI builds [111]. **Obstacles in Fully Automatic Program Repair** identifies overfitting to test suites as a critical barrier, where self-healing maintains runtime function without addressing the root cause [112]. **AdverIntent-Agent** and **Repair-R1** attempt to mitigate this by generating adversarial intents or discriminative tests, but the fundamental reliance on incomplete specifications remains [35], [49]. **Do AI models help produce verified bug fixes?** finds that LLMs do not universally improve debugging performance compared to non-AI groups, especially when formal verification is required [113].

3.4 Multi-Hunk and Repository-Scale Complexity

Most current approaches struggle with bugs requiring coordinated changes across multiple files or functions. **MultiMend** and **SIBLINGREPAIR** make strides in multi-hunk repair by retrieving related code snippets and using semantic matching, yet performance degrades when test suites lack coverage for all sibling locations [90], [114]. **RepoDebug** shows that model performance drops sharply as token length exceeds 500, limiting effectiveness on large repository contexts [115].

Kodezi Chronos attempts to address this with persistent debug memory and graph-guided retrieval, but success rates remain low for distributed system race conditions and hardware-dependent bugs [116].

3.5 Future Directions

Future research should focus on: 1. **Robustness and Generalization:** Developing training objectives and evaluation metrics that penalize overfitting to syntactic patterns and encourage semantic understanding, as suggested by robustness benchmarks [108], [22]. 2. **Efficient Agentic Architectures:** Designing lightweight agent frameworks that balance reasoning depth with computational cost, potentially leveraging smaller, specialized models for specific sub-tasks [117], [118]. 3. **Formal-Neural Hybridization:** Deepening the integration of formal verification and symbolic execution to provide correctness guarantees beyond test suites, moving towards “assured” programming [70], [69]. 4. **Repository-Scale Reasoning:** Advancing context management techniques, such as hierarchical retrieval and graph-based reasoning, to handle cross-file dependencies and large codebases effectively [24], [25]. 5. **Human-in-the-Loop Collaboration:** Exploring interactive debugging assistants that augment rather than replace human developers, leveraging LLMs for hypothesis generation while retaining human oversight for complex semantic decisions [113], [119].

4. Conclusion

The integration of Large Language Models into Automated Program Repair has ushered in a new era of capability, enabling the synthesis of complex patches and the automation of debugging workflows that were previously infeasible. Through retrieval augmentation, agentic reasoning, execution guidance, and specialized fine-tuning, modern systems have demonstrated the ability to fix hundreds of bugs in standard benchmarks and even resolve real-world issues in large repositories. However, the field stands at a crossroads. While generative power has increased, challenges related to robustness, efficiency, and correctness verification remain formidable. The tendency of models to overfit to test suites, struggle with multi-hunk defects, and incur high computational costs necessitates a shift from purely generative approaches to hybrid neuro-symbolic frameworks and rigorous, leakage-free evaluation methodologies. As research progresses towards more autonomous and reliable debugging agents, the synergy between neural intuition and symbolic rigor will likely define the next generation of software maintenance tools.

References

- [1] Kai Huang, Zhengzi Xu, Su Yang. (2022). A Survey on Automated Program Repair Techniques. arXiv:2303.18184.
- [2] Xiang Gao, Yannic Noller, Abhik Roychoudhury. (2022). Program Repair. arXiv:2211.12787.
- [3] Chunqiu Steven Xia, Yuxiang Wei, Lingming Zhang. (2023). Practical Program Repair in the Era of Large Pre-trained Language Models. arXiv:2210.14179.
- [4] Julian Aron Prenner, Romain Robbes. (2021). Automatic Program Repair with OpenAI’s Codex: Evaluating QuixBugs. arXiv:2111.03922.
- [5] Nan Jiang, Kevin Liu, Thibaud Lutellier. (2023). Impact of Code Language Models on Automated Program Repair. arXiv:2302.05020.

- [6] Dominik Sobania, Martin Briesch, Carol Hanna. (2023). An Analysis of the Automatic Bug Fixing Performance of ChatGPT. arXiv:2301.08653.
- [7] Shaunak Samant. (2024). Syntax Is Not Enough: An Empirical Study of Small Transformer Models for Neural Code Repair. arXiv:2512.22216.
- [8] Michael R. Lyu, Baishakhi Ray, Abhik Roychoudhury. (2024). Automatic Programming: Large Language Models and Beyond. arXiv:2405.02213.
- [9] Weishi Wang, Yue Wang, Shafiq Joty. (2023). RAP-Gen: Retrieval-Augmented Patch Generation with CodeT5 for Automatic Program Repair. arXiv:2309.06057.
- [10] Hanyang Guo, Xiaoheng Xie, Hong-Ning Dai. (2025). Accelerating Automatic Program Repair with Dual Retrieval-Augmented Fine-Tuning and Patch Generation on Large Language Models. arXiv:2507.10103.
- [11] Shunyu Liu, Guangdong Bai, Mark Utting. (2024). RELREPAIR: ENHANCING AUTOMATED PROGRAM REPAIR BY RETRIEVING RELEVANT CODE. arXiv:2509.16701.
- [12] Runxiang Cheng, Michele Tufano, Jürgen Cito. (2025). Agentic Bug Reproduction for Effective Automated Program Repair at Google. arXiv:2502.01821.
- [13] Yuwei Zhang, Zhi Jin, Ying Xing. (2024). PATCH: Empowering Large Language Model with Programmer-Intent Guidance and Collaborative-Behavior Simulation for Automatic Bug Fixing. arXiv:2501.16149.
- [14] Qingxiao Xu, Ze Sheng, Zhicheng Chen. (2024). A Systematic Study of LLM-Based Architectures for Automated Patching. arXiv:2603.01257.
- [15] Sungmin Kang, Bei Chen, Shin Yoo. (2024). Explainable Automated Debugging via Large Language Model-driven Scientific Debugging. arXiv:2304.02195.
- [16] Islem Bouzenia, Yangruibo Ding, Kexin Pei. (2024). TraceFixer: Execution Trace-Guided Program Repair. arXiv:2304.12743.
- [17] Kyla H. Levin, Nicolas van Kempen, Emery D. Berger. (2025). ChatDBG: Augmenting Debugging with Large Language Models. arXiv:2403.16354.
- [18] Boyang Yang, Zijian Cai, Shunfu Jin. (2024). PAFT: Preservation-Aware Fine-Tuning for Minimal-Edit Program Repair. arXiv:2604.03113.
- [19] André Silva, Sen Fang, Martin Monperrus. (2024). RepairLLaMA: Efficient Representations and Fine-Tuned Adapters for Program Repair. arXiv:2312.15698.
- [20] David de-Fitero-Dominguez, Antonio Garcia-Cabot, Eva Garcia-Lopez. (2025). Synthetic Code Surgery: Repairing Bugs and Vulnerabilities with LLMs and Synthetic Data. arXiv:2505.07372.
- [21] Runchu Tian, Yinxu Pan, Yesai Wu. (2024). DebugBench: Evaluating Debugging Capability of Large Language Models. arXiv:2401.04621.
- [22] Fengjie Li, Jiajun Jiang, Jiajun Sun. (2024). Evaluating the Generalizability of LLMs in Automated Program Repair. arXiv:2503.09217.
- [23] Wang Bill Zhu, Miaosen Chai, Shangshang Wang. (2025). Precise Debugging Benchmark: Is Your Model Debugging or Regenerating?. arXiv:2604.17338.

- [24] Zhongqiang Pan, Chuanyi Li, Wenkang Zhong. (2018). RepoRepair: Leveraging Code Documentation for Repository-Level Automated Program Repair. arXiv:2603.01048.
- [25] Boyang Yang, Haoye Tian, Jiadong Ren. (2024). Enhancing Repository-Level Software Repair via Repository-Aware Knowledge Graphs. arXiv:2503.21710.
- [26] Chunqiu Steven Xia, Yifeng Ding, Lingming Zhang. (2024). Revisiting the Plastic Surgery Hypothesis via Large Language Models. arXiv:2303.10494.
- [27] Anmol Gautam, Kishore Kumar, Adarsh Jha. (2024). SuperCoder2.0: Technical Report on Exploring the feasibility of LLMs as Autonomous Programmer. arXiv:2409.11190.
- [28] Zhenlong Dai, Zhuoluo Zhao, Hengning Wang. (2025). Learner-Tailored Program Repair: A Solution Generator with Iterative Edit-Driven Retrieval Enhancement. arXiv:2601.08545.
- [29] Spandan Garg, Roshanak Zilouchian Moghaddam, Neel Sundaresan. (2024). RAPGen: An Approach for Fixing Code Inefficiencies in Zero-Shot. arXiv:2306.17077.
- [30] Qianhui Zhao, Fang Liu, Li Zhang. (2024). Peer-aided Repairer: Empowering Large Language Models to Repair Advanced Student Assignments. arXiv:2404.01754.
- [31] Chunqiu Steven Xia, Lingming Zhang. (2023). Keep the Conversation Going: Fixing 162 out of 337 bugs for \$0.42 each using ChatGPT. arXiv:2304.00385.
- [32] Xin Yin, Chao Ni, Shaohua Wang. (2024). ThinkRepair: Self-Directed Automated Program Repair. arXiv:2407.20898.
- [33] Dávid Hidvégi, Khashayar Etemadi, Sofia Bobadilla. (2024). CigaR: Cost-efficient Program Repair with LLMs. arXiv:2402.06598.
- [34] Spandan Garg, Yufan Huang. (2026). Debug2Fix: Supercharging Coding Agents with Interactive Debugging Capabilities. arXiv:2602.18571.
- [35] He Ye, Aidan Z.H. Yang, Chang Hu. (2025). Adversarial Reasoning for Repair Based on Inferred Program Intent. arXiv:2505.13008.
- [36] Nikta Akbarpour, Mahdih Sadat Benis, Fatemeh Hendijani Fard. (2025). Collaborative Agents for Automated Program Repair in Ruby. arXiv:2511.03925.
- [37] Jiangping Huang, Wenguang Ye, Weisong Sun. (2026). TraceCoder: A Trace-Driven Multi-Agent Framework for Automated Debugging of LLM-Generated Code. arXiv:2602.06875.
- [38] Qianru Meng, Zhaochun Ren, Xiao Zhang. (2025). EVIACT: An Evidence-to-Action Framework for Agentic Program Repair. arXiv:2605.27238.
- [39] Haichuan Hu, Congqing He, Hao Zhang. (2024). APRMCTS: Improving LLM-based Automated Program Repair with Iterative Tree Search. arXiv:2507.01827.
- [40] Yixuan Liang. (2024). Monte Carlo Tree Search for Execution-Guided Program Repair with Large Language Models. arXiv:2602.00129.
- [41] Quanming Liu, Xupeng Bu, Zhichao Yan. (2024). T3: Multi-level Tree-based Automatic Program Repair with Large Language Models. arXiv:2506.21211.
- [42] Jialin Song, Jonathan Raiman, Bryan Catanzaro. (2024). Effective Large Language Model Debugging with Best-first Tree Search. arXiv:2407.19055.

- [43] Zhili Huang, Ling Xu, Chao Liu. (2024). DynaFix: Iterative Automated Program Repair Driven by Execution-Level Dynamic Information. arXiv:2512.24635.
- [44] Li Zhong, Zilong Wang, Jingbo Shang. (2024). Debug like a Human: A Large Language Model Debugger via Verifying Runtime Execution Step by Step. arXiv:2402.16906.
- [45] Xueyu Hu, Kun Kuang, Jiankai Sun. (2024). LEVERAGING PRINT DEBUGGING TO IMPROVE CODE GENERATION IN LARGE LANGUAGE MODELS. arXiv:2401.05319.
- [46] Zhengyao Liu, Yunlong Ma, Jingxuan Xu. (2025). Agent That Debugs: Dynamic State-Guided Vulnerability Repair. arXiv:2504.07634.
- [47] Xueying Du, Mingwei Liu, Hanlin Wang. (2018). Exploring Large Language Models in Resolving Environment-Related Crash Bugs: Localizing and Repairing. arXiv:2312.10448.
- [48] Jiaolong Kong, Mingfei Cheng, Xiaofei Xie. (2024). ContrastRepair: Enhancing Conversation-Based Automated Program Repair via Contrastive Test Case Pairs. arXiv:2403.01971.
- [49] Haichuan Hu, Xiaochen Xie, Quanjun Zhang. (2024). Repair-R1: Better Test Before Repair. arXiv:2507.22853.
- [50] Nabiha Parvez, Tanvin Sarkar Pallab, Mia Mohammad Imran. (2026). DePro: Understanding the Role of LLMs in Debugging Competitive Programming Code. arXiv:2603.19399.
- [51] Dekun Dai, MingWei Liu, Anji Li. (2024). FeedbackEval: A Benchmark for Evaluating Large Language Models in Feedback-Driven Code Repair Tasks. arXiv:2504.06939.
- [52] Boyang Yang, Haoye Tian, Jiadong Ren. (2024). MOREpair: Teaching LLMs to Repair Code via Multi-Objective Fine-tuning. arXiv:2404.12636.
- [53] Guochang Li, Chen Zhi, Jialiang Chen. (2024). Exploring Parameter-Efficient Fine-Tuning of Large Language Model on Automated Program Repair. arXiv:2406.05639.
- [54] Wenqiang Luo, Jacky Wai Keung, Boyang Yang. (2024). When Fine-Tuning LLMs Meets Data Privacy: An Empirical Study of Federated Learning in LLM-Based Program Repair. arXiv:2412.01072.
- [55] André Silva, João F. Ferreira, He Ye. (2023). MUFIN: Improving Neural Repair Models with Back-Translation. arXiv:2304.02301.
- [56] Michihiro Yasunaga, Percy Liang. (2021). Break-It-Fix-It: Unsupervised Learning for Program Repair. arXiv:2106.06600.
- [57] He Ye, Matias Martinez, Xiapu Luo. (2022). SelfAPR: Self-supervised Program Repair with Test Execution Diagnostics. arXiv:2203.12755.
- [58] Minh V.T. Pham, Huy N. Phan, Hoang N. Phan. (2025). SWE-Synth: Synthesizing Verifiable Bug-Fix Data to Enable Large Language Models in Resolving Real-World Bugs. arXiv:2504.14757.
- [59] Yuze Zhao, Zhenya Huang, Yixiao Ma. (2024). RePair: Automated Program Repair with Process-based Feedback. arXiv:2408.11296.
- [60] Nafis Tanveer Islam, Mohammad Bahrami Karkevandi, Peyman Najafirad. (2024). Code Security Vulnerability Repair Using Reinforcement Learning with Large Language Models. arXiv:2401.07031.

- [61] Seungjun Moon, Hyungjoo Chae, Yongho Song. (2024). COFFEE: Boost Your Code LLMs by Fixing Bugs with Feedback. arXiv:2311.07215.
- [62] Changxin Ke, Rui Zhang, Jiaming Guo. (2025). QiMeng-PRepair: Precise Code Repair via Edit-Aware Reward Optimization. arXiv:2604.05963.
- [63] Yuanpeng Li, Qi Long, Zhiyuan Yao. (2024). BitsAI-Fix: LLM-Driven Approach for Automated Lint Error Resolution in Practice. arXiv:2508.03487.
- [64] Matthew Jin, Syed Shahriar, Michele Tufano. (2023). InferFix: End-to-End Program Repair with LLMs over Retrieval-Augmented Prompts. arXiv:2303.07263.
- [65] Fengjie Li, Jiajun Jiang, Jiajun Sun. (2024). Hybrid Automated Program Repair by Combining Large Language Models and Program Analysis. arXiv:2406.00992.
- [66] Katsuki Yamagishi, Norihiro Yoshida, Erina Makihara. (2024). Automated Program Repair Based on REST API Specifications Using Large Language Models. arXiv:2510.25148.
- [67] Berkay Berabi, Alexey Gronskey, Veselin Raychev. (2024). DeepCode AI Fix: Fixing Security Vulnerabilities with Large Language Models. arXiv:2402.13291.
- [68] Naman Jain, Shubham Gandhi, Atharv Sonwane. (2024). StaticFixer: From Static Analysis to Static Repair. arXiv:2307.12465.
- [69] Norbert Tihanyi, Ridhi Jain, Mohamed Amine Ferrag. (2024). A New Era in Software Security: Towards Self-Healing Software via Large Language Models and Formal Verification. arXiv:2305.14752.
- [70] Martin Mirchev, Andreea Costea, Abhishek Kr Singh. (2024). Assured Automatic Programming via Large Language Models. arXiv:2410.18494.
- [71] Pedro Orvalho, Mikoláš Janota, Vasco Manquinho. (2024). Counterexample Guided Program Repair Using Zero-Shot Learning and MaxSAT-based Fault Localization. arXiv:2502.07786.
- [72] Sungmin Kang, Haifeng Ruan, Abhik Roychoudhury. (2025). AutoCodeSherpa: Symbolic Explanations in AI Coding Agents. arXiv:2507.22414.
- [73] Vikram Nitin, Rahul Krishna, Luiz Lemos do Valle. (2024). C2SAFERRUST: Transforming C Projects into Safer Rust with NeuroSymbolic Techniques. arXiv:2501.14257.
- [74] Qihao Zhu, Zeyu Sun, Yuan-an Xiao. (2021). A Syntax-Guided Edit Decoder for Neural Program Repair. arXiv:2106.08253.
- [75] Yaojie Hu, Xingjian Shi, Qiang Zhou. (2022). FIX BUGS WITH TRANSFORMER THROUGH A NEURALSymbolic EDIT GRAMMAR. arXiv:2204.06643.
- [76] Sungmin Kang, Shin Yoo. (2022). GLAD: Neural Predicate Synthesis to Repair Omission Faults. arXiv:2204.06771.
- [77] Rohan Bavishi, Harshit Joshi, José Cambronero. (2023). Neurosymbolic Repair for Low-Code Formula Languages. arXiv:2207.11765.
- [78] Zizhang Luo, Yansong Xu, Runlin Guo. (2024). Clover: A Neural-Symbolic Agentic Harness with Stochastic Tree-of-Thoughts for Verified RTL Repair. arXiv:2604.17288.
- [79] Zimin Chen, Steve Kommrusch, Martin Monperrus. (2022). Neural Transfer Learning for Repairing Security Vulnerabilities in C Code. arXiv:2104.08308.

- [80] Mohamad Fakih, Rahul Dharmaji, Halima Bouzid. (2024). LLM4CVE: Enabling Iterative Automated Vulnerability Repair with Large Language Models. arXiv:2501.03446.
- [81] Jugal Gajjar, Kamalasankari Subramaniakuppasamy, Relsy Puthal. (2024). SecureFixAgent: A Hybrid LLM Agent for Automated Python Static Vulnerability Repair. arXiv:2509.16275.
- [82] Sicheol Sung, Joonghyuk Hahn, Yo-Sub Han. (2025). Repairing Regex Vulnerabilities via Localization-Guided Instructions. arXiv:2510.09037.
- [83] Griffin Pitts, Aum Pandya, Darsh Rank. (2026). Automated Program Repair of Uncompilable Student Code. arXiv:2510.06187.
- [84] Jialu Zhang, Jose Cambronero, Sumit Gulwani. (2023). Repairing Bugs in Python Assignments Using Large Language Models. arXiv:2209.14876.
- [85] Fang Liu, Zhenwei Liu, Qianhui Zhao. (2024). FastFixer: An Efficient and Effective Approach for Repairing Programming Assignments. arXiv:2410.21285.
- [86] Sebastian Schweikl, Gordon Fraser. (2025). RePurr: Automated Repair of Block-Based Learners’ Programs. arXiv:2504.12445.
- [87] Tung Phung, José Cambronero, Sumit Gulwani. (2024). Generating High-Precision Feedback for Programming Syntax Errors using Large Language Models. arXiv:2302.04662.
- [88] Shukai Liu, Linzheng Chai, Jian Yang. (2024). MDEVAL: Massively Multilingual Code Debugging. arXiv:2411.02310.
- [89] Harshit Joshi, Jose Cambronero, Sumit Gulwani. (2023). Repair Is Nearly Generation: Multilingual Program Repair with LLMs. arXiv:2208.11640.
- [90] Reza Gharibi, Mohammad Hadi Sadreddini, Seyed Mostafa Fakhrahmad. (2024). MULTIMEND: MULTILINGUAL PROGRAM REPAIR WITH CONTEXT AUGMENTATION AND MULTI-HUNK PATCH GENERATION. arXiv:2501.16044.
- [91] Kyle Wong, Alfonso Amayuelas, Liangming Pan. (2024). Investigating the Transferability of Code Repair for Low-Resource Programming Languages. arXiv:2406.14867.
- [92] Ke Xu, Jialin Sun, Yuchen Hu. (2024). MEIC: Re-thinking RTL Debug Automation using LLMs. arXiv:2405.06840.
- [93] Ning Wang, Bingkun Yao, Jie Zhou. (2024). VeriDebug: A Unified LLM for Verilog Debugging via Contrastive Embedding and Guided Correction. arXiv:2504.19099.
- [94] Xufeng Yao, Haoyang Li, Tsz Ho Chan. (2024). HDLdebugger: Streamlining HDL debugging with Large Language Models. arXiv:2403.11671.
- [95] Fengmin Zhu, Fei He. (2024). EqFix: Fixing LATEX Equation Errors by Examples. arXiv:2107.00613.
- [96] Xiaoyu Guo, Jianjun Zhao, Pengzhan Zhao. (2024). On Repairing Quantum Programs Using ChatGPT. arXiv:2401.14913.
- [97] Soneya Binta Hossain, Nan Jiang, Qiang Zhou. (2024). A Deep Dive into Large Language Models for Automated Bug Localization and Repair. arXiv:2404.11595.

- [98] Melika Sepidband, Hamed Taherkhani, Hung Viet Pham. (2026). RGFL: Reasoning Guided Fault Localization for Automated Program Repair Using Large Language Models. arXiv:2601.18044.
- [99] Jianming Chang, Xin Zhou, Lulu Wang. (2024). Bridging Bug Localization and Issue Fixing: A Hierarchical Localization Framework Leveraging Large Language Models. arXiv:2502.15292.
- [100] Yinsheng Yao, Hongxiang Zhang, Weixi Tong. (2025). FLARE: Fine-Grained Diagnostic Feedback for LLM Code Refinement. arXiv:2606.03852.
- [101] Jiaolong Kong, Xiaofei Xie, Yiheng Xiong. (2024). Enhancing Automated Program Repair via Faulty Token Localization and Quality-Aware Patch Refinement. arXiv:2511.18001.
- [102] Melika Sepidband, Hung Viet Pham, Hadi Hemmati. (2026). On the Role of Fault Localization Context for LLM-Based Program Repair. arXiv:2604.05481.
- [103] Joseph Townsend, Chandresh Pravin, Kwun Ho Ngan. (2024). A Study on the Impact of Fault localization Granularity for Repository-Scale Code Repair Tasks. arXiv:2604.00167.
- [104] Julian Aron Prenner, Romain Robbes. (2024). Out of Context: How important is Local Context in Neural Program Repair?. arXiv:2312.04986.
- [105] Julian Aron Prenner, Romain Robbes. (2024). Extracting Fix Ingredients using Language Models. arXiv:2503.04214.
- [106] André Silva, Martin Monperrus. (2024). RepairBench: Leaderboard of Frontier Models for Program Repair. arXiv:2409.18952.
- [107] Quanjun Zhang, Tongke Zhang, Juan Zhai. (2024). A Critical Review of Large Language Model on Software Engineering: An Example from ChatGPT and Automated Program Repair. arXiv:2310.08879.
- [108] Fazle Rabbi, Jinqiu Yang. (2026). HEJ-Robust: A Robustness Benchmark for LLM-Based Automated Program Repair. arXiv:2605.02215.
- [109] Anvith Pabba, Simin Chen, Alex Mathai. (2024). REFINE: Enhancing Program Repair Agents through Context-Aware Patch Refinement. arXiv:2510.03588.
- [110] Zhenyu Yang, Zhen Yang, Zhongxing Yu. (2024). NARRepair: Non-Autoregressive Code Generation Model for Automatic Program Repair. arXiv:2406.16526.
- [111] Davide Ginelli, Matias Martinez, Leonardo Mariani. (2024). A Comprehensive Study of Code-removal Patches in Automated Program Repair. arXiv:2012.06264.
- [112] S. Amirhossein Mousavi, Donya Azizi Babani, Francesco Flammini. (2024). Obstacles in Fully Automatic Program Repair: A survey. arXiv:2011.02714.
- [113] Li Huang, Ilgiz Mustafin, Marco Piccioni. (2024). Do AI models help produce verified bug fixes?. arXiv:2507.15822.
- [114] Xinyu Liu, Jiayu Ren, Yusen Wang. (2024). SIBLINGREPAIR: Sibling-Based Multi-Hunk Repair with Large Language Models. arXiv:2605.06209.
- [115] Jingjing Liu, Zeming Liu, Zihao Cheng. (2025). RepoDebug: Repository-Level Multi-Task and Multi-Language Debugging Evaluation of Large Language Models. arXiv:2509.04078.

- [116] Ishraq Khan, Assad Chowdary, Sharoz Haseeb. (2025). Kodezi Chronos: A Debugging-First Language Model for Repository-Scale Code Understanding. arXiv:2507.12482.
- [117] David Jiahao Fu, Aryan Gupta, Aaron Councilman. (2025). SLMFIX: LEVERAGING SMALL LANGUAGE MODELS FOR ERROR FIXING WITH REINFORCEMENT LEARNING. arXiv:2511.19422.
- [118] Matheus J. T. Vargas. (2024). SLEAN: Simple Lightweight Ensemble Analysis Network for Multi-Provider LLM Coordination: Design, Implementation, and Vibe Coding Bug Investigation Case Study. arXiv:2510.10010.
- [119] Boyang Yang, Haoye Tian, Weiguo Pian. (2024). Cref: An LLM-based Conversational Software Repair Framework for Programming Tutors. arXiv:2406.13972.